

dea-tools: Geospatial analysis of satellite data using Xarray, the Open Data Cube, and Digital Earth Australia

Chad Burton¹✉, Robbi Bishop-Taylor¹, Bex Dunn¹, Claire Phillips¹, Caitlin Adams¹, Gianluca Scortechini¹, Sean Chua¹, Vanessa Newey¹, Margaret Harrison¹, Jenna Guffogg¹, Claire Krause¹, and James Miller¹

¹ Geoscience Australia, Australia  ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

dea-tools is an open-source Python library that provides a comprehensive set of tools for analysing, visualising, and modelling geospatial and Earth observation data represented as xarray objects. Originally developed to support Digital Earth Australia (DEA) workflows, dea-tools now functions as a general-purpose library that can be used with any geospatial dataset. The package offers high-level utilities for spatial and temporal analysis, remote sensing index calculation, machine learning, and interactive visualisation, while integrating seamlessly with the Python geospatial stack (e.g. xarray, geopandas, numpy, and dask). Distributed as a pip-installable environment, dea-tools includes JupyterLab and related dependencies, providing a fully managed platform for scientific geospatial analysis on any machine. Embedded within the broader dea-notebooks repository, the package is supported by extensive documentation and example Jupyter notebooks demonstrating use of dea-tools functionality. Together, dea-tools and dea-notebooks enable reproducible, scalable, and accessible Earth observation science with Python.

Statement of need

Modern Earth observation workflows often involve complex, multi-temporal, and multi-resolution datasets that demand integrated tools for efficient processing, analysis, and visualisation. While foundational libraries such as xarray, geopandas, and dask provide the building blocks, implementing complex analysis workflows still requires substantial coding effort to handle data loading and harmonisation, visualisation, spatio-temporal analysis, and modelling. The dea-tools library addresses this gap by providing a curated suite of high-level, well-documented functions that seamlessly integrate with these core libraries.

The dea-tools Python package provides a comprehensive, open-source toolkit for the analysis, and interpretation of geospatial and Earth observation (EO) data represented primarily as xarray objects. Developed by Digital Earth Australia (DEA) to streamline analysis of national-scale satellite datasets, dea-tools has since evolved into a flexible and general-purpose library that supports many geospatial workflows. It can now be used on any system and with any geospatial dataset, whether accessed from cloud-based catalogues via [odc-stac](#) or stored locally.

A major advantage of dea-tools is it comes bundled with all essential dependencies for scientific geospatial analysis, including JupyterLab, Jupyter notebooks, xarray, geopandas, numpy, dask, and other related ecosystem libraries. When deployed within a virtual environment, dea-tools therefore provides a **fully managed and ready-to-use geospatial analysis platform**. This design

lowers the barrier to entry for new users, enabling a self-contained environment for exploration, visualisation, and large-scale satellite data processing.

The package enables users to:

- Load, combine, and manage satellite or model datasets, including but not limited to DEA products, while handling reprojection and resolution harmonisation;
- Conduct spatial and temporal analyses directly on xarray objects (e.g., extracting temporal statistics, linear regression, and interpolation);
- Apply a large curated list of remote sensing indices such as NDVI, NDWI etc.
- Perform machine learning and segmentation workflows on geospatial datasets, with built-in training, fitting, and prediction tools that work seamlessly with xarray;
- Generate publication-quality visualisations and animations;
- Seamlessly convert between raster and vector formats using xarray–geopandas interoperability utilities.
- Apply domain-specific tools: Analyse coastal change, intertidal zones, land cover, wetland and waterbody dynamics, and climate datasets.

The broader dea-notebooks repository, which includes dea-tools, provides extensive documentation as Jupyter notebooks including how-to guides and longer ‘real-world’ application examples. These include end-to-end workflows that demonstrate how to acquire satellite data via [odc-stac](#), process it using dea-tools, and visualize outputs interactively within JupyterLab. Together, dea-tools and dea-notebooks offer a reproducible, and fully managed framework for geospatial analysis in Python — bridging the gap between foundational geospatial libraries and applied EO research.

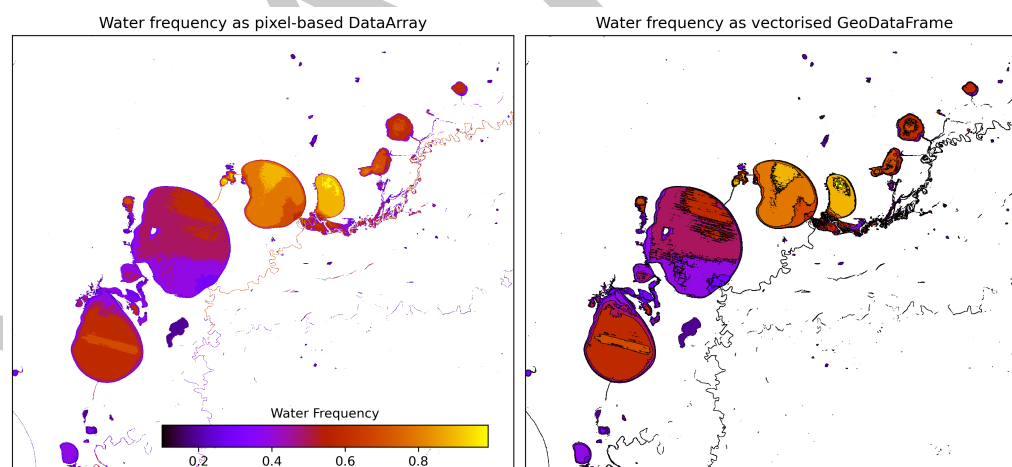


Figure 1: dea-tools enables easy transitions between xarray and geopandas with the `xr_rasterize` and `xr_vectorize` functions.

Features

Below we highlight a non-exhaustive list of functionality within dea-tools. Note that each of the functions listed below has an accompanying Jupyter notebook outlining how to use the function.

Analytical tools native to Xarray

Flexible and powerful functionality for scientific analysis with xarray objects:

- Extracting time series statistics, using the module [dea-tools.temporal](#):

- 69 – Generic summary statistics on any time series with `temporal_statistics`
- 70 – Phenology with `xr_phenology`
- 71 – Linear regression with `xr_regression`
- 72 ■ Validation tools within [dea-tools.validation](#):
- 73 – Compute common statistical metrics with `eval_metric`
- 74 – Generate random points over an xarray object with `xr_random_sampling`

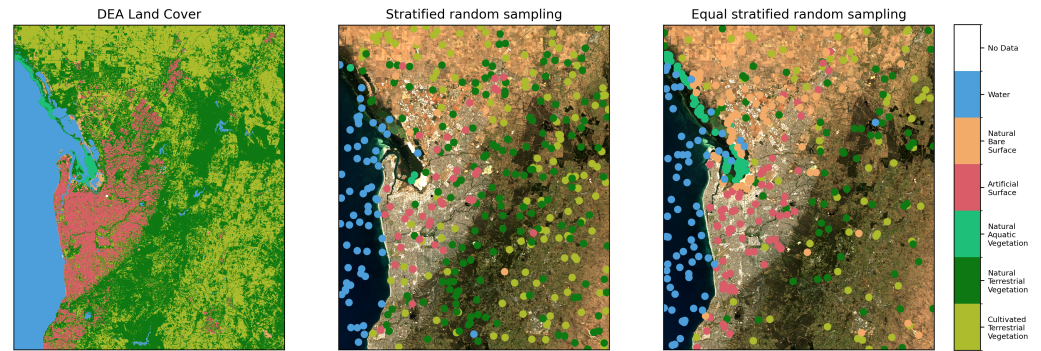


Figure 2: Example extraction of random samples from a classified `xarray.DataArray` using two of the sampling strategies available in the `xr_random_sampling` function.

- 75 ■ Spatial analysis tools within [dea-tools.spatial](#):
- 76 – Vectorise `xarray.DataArrays` into `geopandas.GeoDataFrames` with `xr_vectorise`
- 77 – Rasterize `geopandas.GeoDataFrames` into `xarray.DataArrays` with `xr_rasterize`
- 78 – Extract subpixel contours from `xarray.DataArrays` with `subpixel_contours`
- 79 – Interpolate point data stored in a `geopandas.GeoDataFrame` into an `xarray.DataArray` with `xr_interpolate`
- 80
- 81 ■ Xarray wrappers for the full machine learning pipeline in [dea-tools.classification](#):
- 82 – prepare data for ML predictions with `sklearn_flatten` and `sklearn_unflatten`
- 83 – Fit models with `fit_xr`
- 84 – ML predictions with `predict_xr`
- 85 – Handle spatial autocorrelation in cross-validation workflows with `spatial_train_test_split`
- 86 and `SKCV` (spatial k-fold cross validation)

87 Data handling

88 Tools for loading and manipulating both DEA satellite data, along with other providers of
89 geospatial data, stored within [dea-tools.datahandling](#)

- 90 ■ `load_ard`
- 91 ■ `load_reproject`
- 92 ■ `parallel_apply`

93 Plotting

94 The [dea-tools.plotting](#) library has extensive functionality for generating beautiful imagery from
95 xarray objects.

- 96 ■ Highly customisable animations with `xr_animation`
- 97 ■ RGB plots for true and false colour composite images with `rgb`

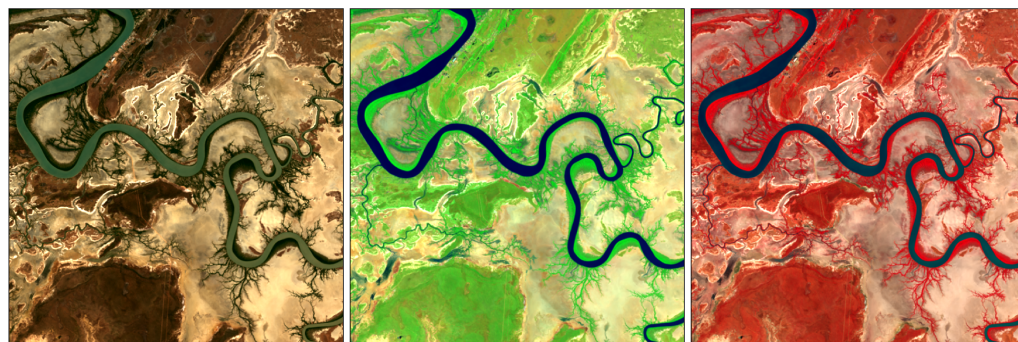


Figure 3: True and false colour composites generated with the `dea-tools.plotting.rgb` function, using DEA annual GeoMAD composite images from 2024.

Research projects

dea-tools and the broader dea-notebooks repository supports a wide range of scientific applications, from agriculture and land cover mapping to wetland, coastal, and surface water monitoring, and climate and fire analysis. The package has already underpinned more than 25 peer-reviewed studies, demonstrating its robustness and adaptability across domains. Usage of the library is recorded [here](#).

Acknowledgements

Thanks

References